

Let Lean 4 be Falcon's referee.

An auditable middleware for Arabic math & logic conversations: Falcon answers → formalize → Lean kernel decides → grounded correction → Falcon revises.

Falcon-H1-Arabic-3B on 86 Arabic math/logic problems: accuracy 58% → 88%, 0 answers turned from right to wrong. Code, data and every per-problem trace are open.

LLMs are confidently wrong — more so in Arabic

and checking a chain of reasoning by hand is expensive

Why Falcon, why Arabic

- Falcon is TII's home-grown model; local users ask it in Arabic.
- Its pre-training corpus is mostly English → weaker Arabic math and logic.
- Same problems: Falcon 7B 97%, Falcon Arabic-3B only 58%.
- Arabic itself amplifies risk: two digit systems (٩٠٠ vs 0-9), RTL text mixed with numbers, rich morphology, free word order, ambiguous «إما...أو» (either/or).

Typical errors (real 3B output)

ثم $720 = 0.9 \times 800$ ، $576 = 0.8 \times 720$... الجواب النهائي:
680

"800 × 0.9 = 720, then 720 × 0.8 = 576 ... Final answer: 680"

Every step right, final answer wrong — a reader who only checks the conclusion is fooled.

عمر أطول من يوسف، ويوسف أطول من خالد. هل يلزم أن عمر أطول
من خالد؟ → لا

"Omar is taller than Yusuf, Yusuf taller than Khalid. Must Omar be taller than Khalid?" → "No"

Transitivity answered backwards — fluent language, broken logic.

Our goal

not a smarter model — a referee that cannot lie

What we do

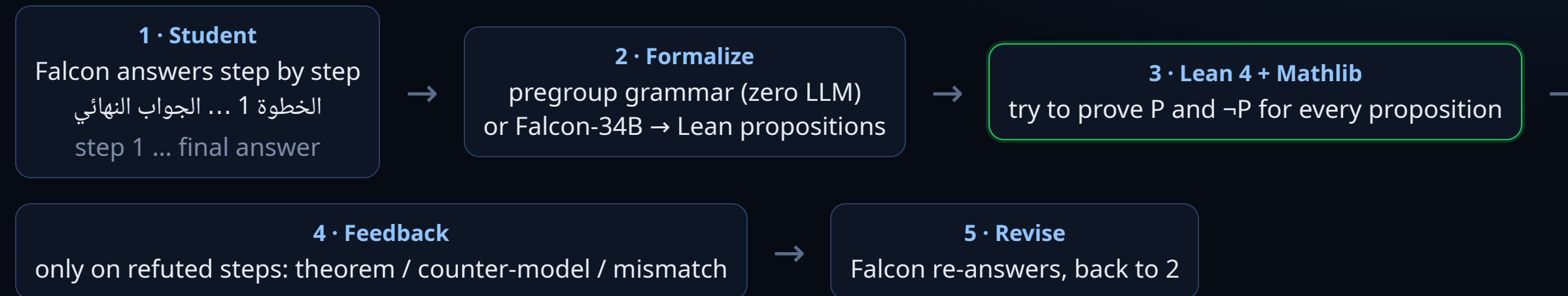
- **Decide truth:** translate each reasoning step and the final answer into Lean 4 propositions; the theorem-prover kernel decides.
- **Teach:** only steps proven false get feedback, grounded in a Lean theorem or a concrete counter-model; Falcon answers again.
- **Leave evidence:** every answer ships with an assurance trace — propositions, verdicts, feedback, revisions.

What we refuse to do

- Never treat an LLM judgment as evidence: both LLMs (student, translator) are generators; **Lean is the only truth authority.**
- Never fake verification: outside the decidable fragment we honestly say `unknown` instead of producing a more confident text.
- No weight changes today — an external referee now, a training signal next.

Workflow: the verify-teach loop

Falcon never sees Lean; it only sees a teacher's natural-language correction



Five verdicts: **verified** Lean proved P · **refuted** Lean proved $\neg P$ · **unknown** neither · **ill_formed** does not type-check · **skipped** no checkable claim

Guards against false alarms: $\mathbb{Q}$ re-check, number grounding, degenerate-reasoning filter, structural round-trip signature (relation arity / symmetry / counts missing → downgraded to unknown). 10–40 s per round.

Math background ①: pregroup grammar (Lambek 1999)

“is this sentence well-formed?” becomes “do the types reduce to s?”

Three rules are enough

Each word gets a type built from basic types n (noun / number), s (sentence), q (question) and their left/right adjoints x^l , x^r .

$x^l \cdot x \rightarrow 1$ $x \cdot x^r \rightarrow 1$ (contraction)
well-formed $\Leftrightarrow$ the product of word types reduces to s

The verb “equals” has type $n^r \ s \ n^l$: it consumes one n on its left, one n on its right, and leaves s .

Arabic example

٣٩١ يساوي ٢٣ × ١٧

“17 × 23 equals 391” (Eastern Arabic digits)

$n \cdot n^r \ n \ n^l \cdot n \cdot n^r \ s \ n^l \cdot n$
 $= n \cdot (n^r \ n \ n^l) \cdot n \rightarrow n$ (17×23 is a number)
 $n \cdot n^r \ s \ n^l \cdot n \rightarrow s$ (“equals” eats one n on each side)

The reduction path is itself a **certificate**, stored in every trace: pregroup: $n \ n^r \ s \ n^l \ n \rightarrow s$. The same path drives the Lean proposition: $(17:\mathbb{Q}) * 23 = 391$.

Math background ②: syntax $\rightarrow$ semantics as a functor

DisCoCat (Coecke–Sadrzadeh–Clark 2010)

Category theory in one breath

- A pregroup is a **compact closed monoidal category**: word types are objects, contractions $x^L x^R \rightarrow 1$ are morphisms (“cups”).
- Meaning lives in another category — vector spaces, relations, or, here, **Lean propositions**.
- Translation is a structure-preserving **functor F**: $F(\text{reduction}) = \text{how meanings compose}$.

```
F : Grammar  $\rightarrow$  LeanProp
F(n) =  $\mathbb{Q}$  / Fin 3  $\rightarrow$  Bool      F(s) = Prop
F(nr s nL “equals”) = fun a b  $\Rightarrow$  a = b
```

Why this matters

- **Compositionality**: the meaning is fixed by the shape of the reduction; word meanings only appear at the leaves.
- **Word order is irrelevant**: we proved in Lean that Arabic VSO and SVO derive the same meaning (`vso_svo_same_meaning`), so the parser may normalise freely.
- **Provable limits**: coarse types lose gender/number agreement — also a Lean theorem (`coarse_accepts_bad_agreement`), pointing to dependent types as future work.

No need to know what a “square” is to judge the inference

logical validity depends only on the shape of the sentence, not on word meanings

Problem (3B first answers “yes” or omits the answer — wrong either way)

كل المربعات مستطيلات، وبعض المستطيلات ليست مربعات. هل يلزم أن بعض المربعات ليست مستطيلات؟

“All squares are rectangles, and some rectangles are not squares. Must some squares not be rectangles?”

pregroup: $B \text{ ليست } A \vdash ? \text{ بعض } B \text{ ليست } A$ كل $A \ B$
A := مربعات $\equiv$ المربعات (morphological variants merged, in the certificate)
B := مستطيلات $\equiv$ المستطيلات

Lean: $\forall (A \ B : \text{Fin } 3 \rightarrow \text{Bool}),$
 $(\forall x, A \ x \rightarrow B \ x) \rightarrow (\exists x, B \ x \wedge \neg A \ x) \rightarrow \exists x, A \ x \wedge \neg B \ x$

Lean's answer

refuted — `decide` finds a counter-model: $A = \{0\}$, $B = \{0, 1\}$.

مثال مضاد: $\{x0\} = \text{«المربعات»}$, $\{x0, x1\} = \text{«مستطيلات»}$. كل المقدمات صحيحة ولكن «بعض المربعات ليست مستطيلات» خاطئة.

Feedback sent to Falcon: “Counter-example: squares = {x0}, rectangles = {x0, x1}. All premises hold, but ‘some squares are not rectangles’ is false.”
→ 3B revises to «لا» (“no”).

- Lean contains no “square” and no “rectangle” — only predicates A and B. **The verdict is independent of word meaning.**
- A reviewer who reads no Arabic can still audit this: proposition, verdict, counter-model.

Example ②: five students and the handshake lemma

formalization can go wrong too, so we protect the shape

Problem

خمسة طلاب، يقول كل واحد منهم إن ثلاثة من الأربعة الباقين
أصدقاؤه. هل يلزم أن أحدهم يكذب؟

"Five students; each says three of the other four are his friends. Must someone be lying?"

Wrong formalization (LLM translator): $\text{students} : \text{Fin } 5 \rightarrow \text{Bool}$ — friendship squashed into one boolean per person, losing that it is binary and symmetric. Lean correctly refuted a *wrong* proposition $\rightarrow$ false alarm.

Right formalization (deterministic fragment)

$\forall f : \text{Fin } 5 \rightarrow \text{Fin } 5 \rightarrow \text{Bool}, \neg \text{Regular } f \ 3$

$\text{Regular } f \ k := (\forall x \ y, f \ x \ y = f \ y \ x) \quad \text{-- symmetric}$
 $\quad \wedge (\forall x, f \ x \ x = \text{false}) \quad \text{-- no self-loops}$
 $\quad \wedge \forall x, \text{degree } f \ x = k$

verified in 18 s via the handshake lemma $\sum \text{degree} = 2 \cdot |E|$: $5 \times 3 = 15$ is odd $\rightarrow$ contradiction. No enumeration of 2^{25} relations.

Round-trip signature: the question is about a *mutual relation between individuals* with count 3; a formula lacking a binary type, symmetry or the number 3 is downgraded to unknown — no more false alarm.

Example ③: one round of feedback fixes a wrong final answer

Falcon-3B · real trace (armath-048)

Round 1

سعر هاتف 800 درهم، خُفِّض 10% ثم 20%. ما السعر النهائي؟

"A phone costs 800 AED, discounted 10% then 20%. What is the final price?"

step 2	$(800:\mathbb{R}) * 10/100 = 80 \wedge 800 - 80 = 720$	verified
step 3	$(720:\mathbb{R}) * 20/100 = 144 \wedge 720 - 144 = 576$	verified
final	Lean proves $800 * (1 - 0.10) * (1 - 0.20) = 576$	
	Falcon wrote 680	refuted

الجواب النهائي لا يطابق ما تستنتجه خطواتك. تحقق Lean 4 من أن
 $576 = \dots$

Feedback: "Your final answer does not match what your own steps imply. Lean 4 verified that ... = 576."

Round 2

step 2, 3	verified	
final 576	verified	assurance = 1.0

The feedback adds nothing new: it quotes a proposition Falcon itself derived and Lean proved.

This error class drives most of the 3B math gain 54% → 87%; Eastern-digit problems 55% → 84%.

Human review: from “read every step” to “look at the red ones”

formalization turns review from understanding the problem into checking a certificate

72/86

final answers fully Lean-verified and correct — no human reading needed

1

false alarm on a correct answer (2%), causing no regression

0

answers turned from right to wrong

92%

baseline errors caught by Lean (33/36)

- Reviewers only inspect **refuted** and **unknown** items; each carries the Lean proposition and kernel diagnostics and is reproducible (`lake env lean`).
- Checking needs no language skill: propositions are Lean, counter-examples are finite models, certificates are pregroup reductions — an Arabic-illiterate reviewer can audit an Arabic conversation.
- Outside the fragment the system degrades honestly: proof questions, open “why”, and relation problems before the graph fragment existed returned unknown rather than a fake verification.

Benchmark: Falcon Arabic-3B, 86 Arabic problems

paired design: baseline = round 1 of the same trajectory · Lean verdicts + exact-answer match · 0 regressions

slice	n	baseline	verified	caught by Lean	fixed after feedback	false alarms	regressions
all	86	58%	88%	33/36	26/36	1	0
math	52	54%	87%	24/24	17/24	1	0
logic	34	65%	91%	9/12	9/12	0	0
hard (compound discount / order of operations)	6	50%	83%	3/3	2/3	0	0
syllogisms	8	50%	88%	3/4	3/4	0	0
pregroup arithmetic fragment (zero LLM)	16	69%	94%	5/5	4/5	0	0
relation / counting (handshake lemma)	10	70%	100%	3/3	3/3	0	0
Eastern Arabic digits ٩-٠	31	55%	84%	14/14	9/14	1	0

The original 76-problem subset: 55.3% → 84.2% in three independent reruns, exact McNemar $p \approx 4.8 \times 10^{-7}$. Controls: Falcon 7B Arabic 97% → 99%; English 7B 97% → 99% (89 problems), hard 97% → 99% (78). Mean 1.69 rounds, 39 s per problem. Self-built dataset — not directly comparable with GSM8K-style public benchmarks.

Does it hold at scale? 244 freshly generated Arabic problems

same student, same formalizer, harder mix (two-digit × two-digit word problems, compound discounts, relations)

76%

accuracy after Lean feedback
(baseline 48%)

117/127

baseline errors caught by Lean
(92%)

1 · 0

false alarms · regressions

194

final answers Lean proved (185 of
them also match the gold
answer)

- Detection stays above 90% while the student gets weaker (48% baseline): Lean does not care how hard the problem is for Falcon.
- Fix rate drops to 54%: the 3B student sometimes cannot act on a correct correction — a limit of the student, visible and counted, not hidden.
- Six generator gold answers were themselves wrong (rounding); the pipeline flagged 3B as 'wrong' although Lean had verified its arithmetic — we fixed the generator and report the corrected numbers.

Strict assurance: Lean audits the chain of thought

what the verdict badge next to every step means

run	rounds	steps	Lean decided	verified	refuted	unknown	skipped	final answers decided
86 problems	145	395	53%	50%	3%	5%	42%	96%
244 problems	453	1314	55%	51%	4%	3%	42%	98%

Skipped = narrative sentences («لنحسب أولاً» “*let us first compute...*”) that make no checkable claim. They are shown grey, never counted as verified.

Reading a badge

- **verified** Lean proved P · **refuted** Lean proved $\neg P$ · **unknown** neither, within budget · **skipped** no claim to check.
- assurance = $\frac{1}{2}$ verified share of checkable steps + $\frac{1}{2}$ final-answer verdict — a coverage score, not a confidence.
- Correction is a by-product: when Falcon is right in round 1, the same screen is a machine-checked certificate of its reasoning.

What we claim — and what we do not

- **Claim:** for every Falcon round, the kernel returns a proof or a refutation for more than half of the reasoning sentences and for ~96–98% of final answers.
- **Claim:** a green badge means Lean proved *that proposition*; the .lean file sent to the kernel is one click away in the UI (audit panel).
- **Not a claim:** that the proposition captures every nuance of the Arabic sentence — the structural round-trip check narrows, but does not close, that gap.

More deterministic formalization → larger gain

Arabic logic problems, Falcon-3B, same student across versions

formalizer version	n	baseline	verified	caught by Lean	false alarms
① LLM (Falcon-34B) translation only	24	66.7%	66.7%	0/8	0
② + yes/no polarity alignment, degenerate-reasoning filter	24	62.5%	79.2%	3/9	0
③ + pregroup deterministic fragment + counter-model teaching	24	70.8%	91.7%	7/7	0
④ + relation/counting fragment (10 handshake problems added)	34	65%	91%	9/12	0

- The gain comes not from a stronger model but from **taking translation away from the LLM**: pregroup reduction fixes the shape of the proposition, Lean fixes its truth.
- Problems still on the LLM path (outside the fragment) remain the weak link — documented as such.

Why Arabic

largest need, and a regular grammar that makes deterministic formalization feasible

Demand: Arabic needs an external referee most

- Arabic is a minority language in Falcon's pre-training → lower baseline, more hallucination (3B: 58% vs 97% for English / 7B).
- Two digit systems, RTL mixing, clitics fused into words, free word order, ambiguous connectives — all places where meaning is lost during reasoning.
- Mathematical and logical truth is language-independent: once in Lean, the verdict no longer depends on how much Arabic the model saw.

Supply: Arabic lends itself to formalization

- Modern Standard Arabic is a normed written language; quantifier patterns are fixed: كل / بعض / لا (all / some / none / if...then).
- Root-and-pattern morphology is highly regular — a good fit for algebraic grammars and deterministic parsing.
- VSO $\equiv$ SVO is proven in Lean, so word-order normalisation is safe.
- Boundary: morphology makes coarse types lose faithfulness (also proven) — a research direction, not a hidden flaw.

Limits and next steps

honest boundaries, and a data flywheel

What we cannot do today

- Proof questions / open “why”: no decidable final proposition.
- Algebra / age word problems: the LLM mis-translates text into equations; Lean catches 0.
- 3B occasionally refuses to fix an answer Lean refuted (armath-051).
- Latency 39 s per problem; Lean + Mathlib is a heavy deployment.

Next

- **Data flywheel:** every corrected trace is a (prompt, Lean-refuted answer, Lean-verified answer, kernel evidence) preference pair → DPO on open Falcon-H1-3B, internalising the referee. **294 pairs** (226 Arabic / 68 English) exported, no human or LLM judge involved.
- Public benchmarks (GSM8K-ar, MGSM-ar): report accuracy and “decidable coverage”.
- Product: hosted service + Open WebUI plugin (Docker ready); Arabic K-12 math grading.
- Research: agreement features as dependent types; $VSO \equiv SVO$ as a 2-cell between derivations.

What you can touch today

open source · Docker image with Lean + Mathlib · one-line API

Judge demo (5 min, phone browser)

- `/Verifier`: pick an Arabic example — badge shows how often 3B got it wrong in our runs; English gloss appears under the RTL input.
- Watch each step turn **verified** / **refuted** live, with the Lean proposition beside it; open the audit panel to see the exact .lean file the kernel checked.
- Red round → Arabic feedback → green round; download the full assurance trace as JSON.
- No time for a 40 s round? Replay any recorded correction from the benchmark — same renderer, zero model calls.
- **/benchmark**: charts + every problem, gold/baseline/final answer, links to the trace file and grading code on GitHub.

Middleware, not a chatbot

```
curl -X POST $HOST/api/solve -d '{"problem": "...",
"student_model": "falcon-h1-arabic-3b-instruct"}'
→ { final_answer, status, assurance_score, rounds: [ {
  answer,
    report: { steps: [ { text, lean_prop, verdict } ],
  final_answer_verdict },
  feedback } ] }
```

- Any application that already calls Falcon adds one HTTP hop and gets verdicts + an auditable trace back.
- Docker image (Python + Lean 4 + Mathlib cache) published to GHCR on every merge; Caddy HTTPS, rate limiting, access token, health checks in `deploy/`.
- 99 tests + Lean build in CI; every number on these slides is regenerated from `bench/results/*.json`.

Ask the kernel, not the model.

Falcon-3B Arabic: 58% → 88%, 0 regressions, and every correction is backed by a Lean proof.

`github.com/hw4375-collab/FalconVerifier · pip install -e . && falconverifier serve · traces, report and these slides are generated from the raw results.`